

Graphical Models : Discrete Inference & Learning

Style Transfer using Graph Cuts

Antoine DEBOUCHAGE, Thomas LEMERCIER, Gaspard BERTHELIER

State of the art

Style transfer : applying the “style”
(shape, color, texture) of a *style image*
to a *content image*

content image



style image



Two families of methods :

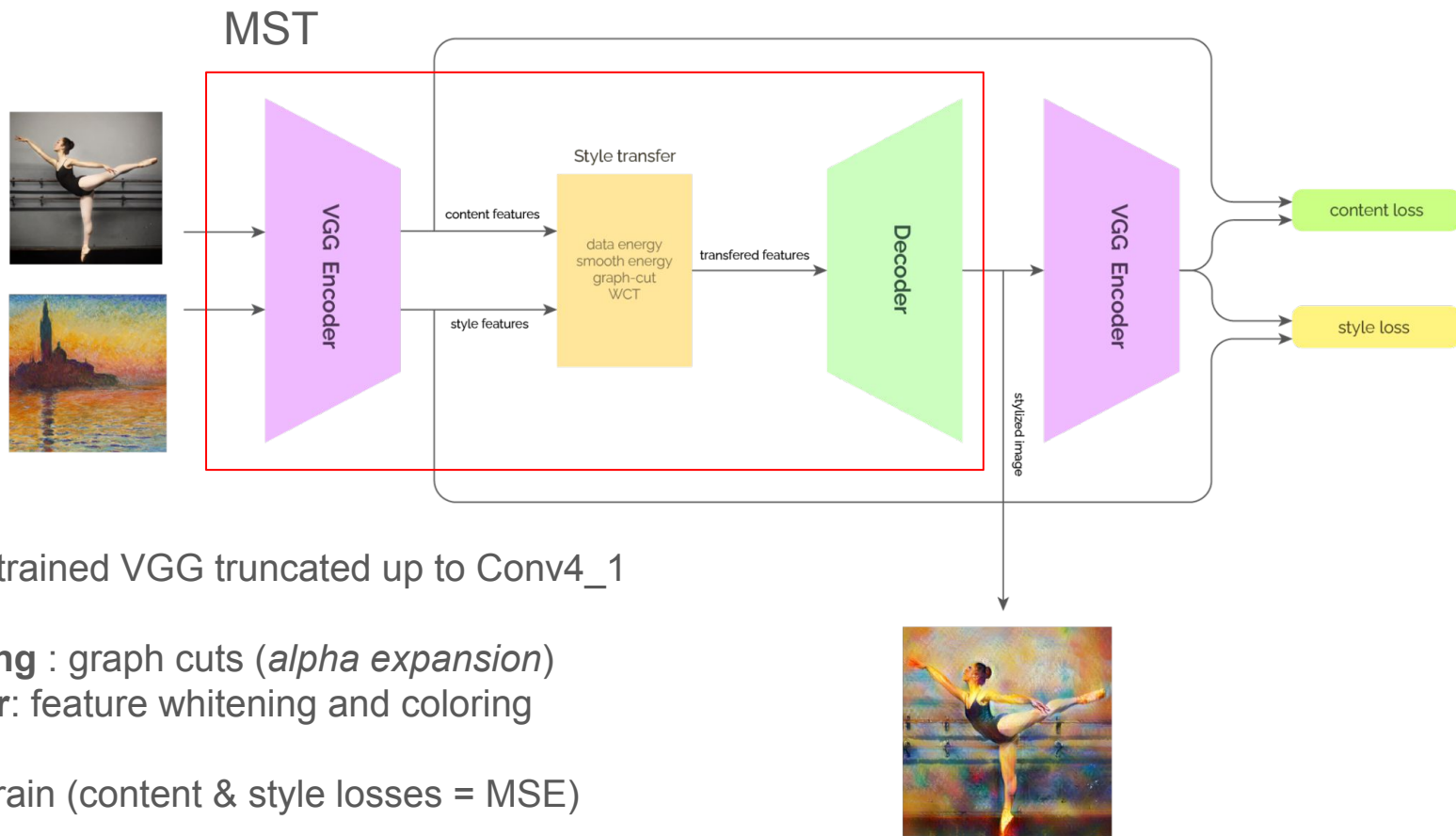
- ❑ Parametric : using the statistics of style image
- ❑ Patch-based : matching and merging patch of pixels

Issues : artifacts, wash-out effect, errors

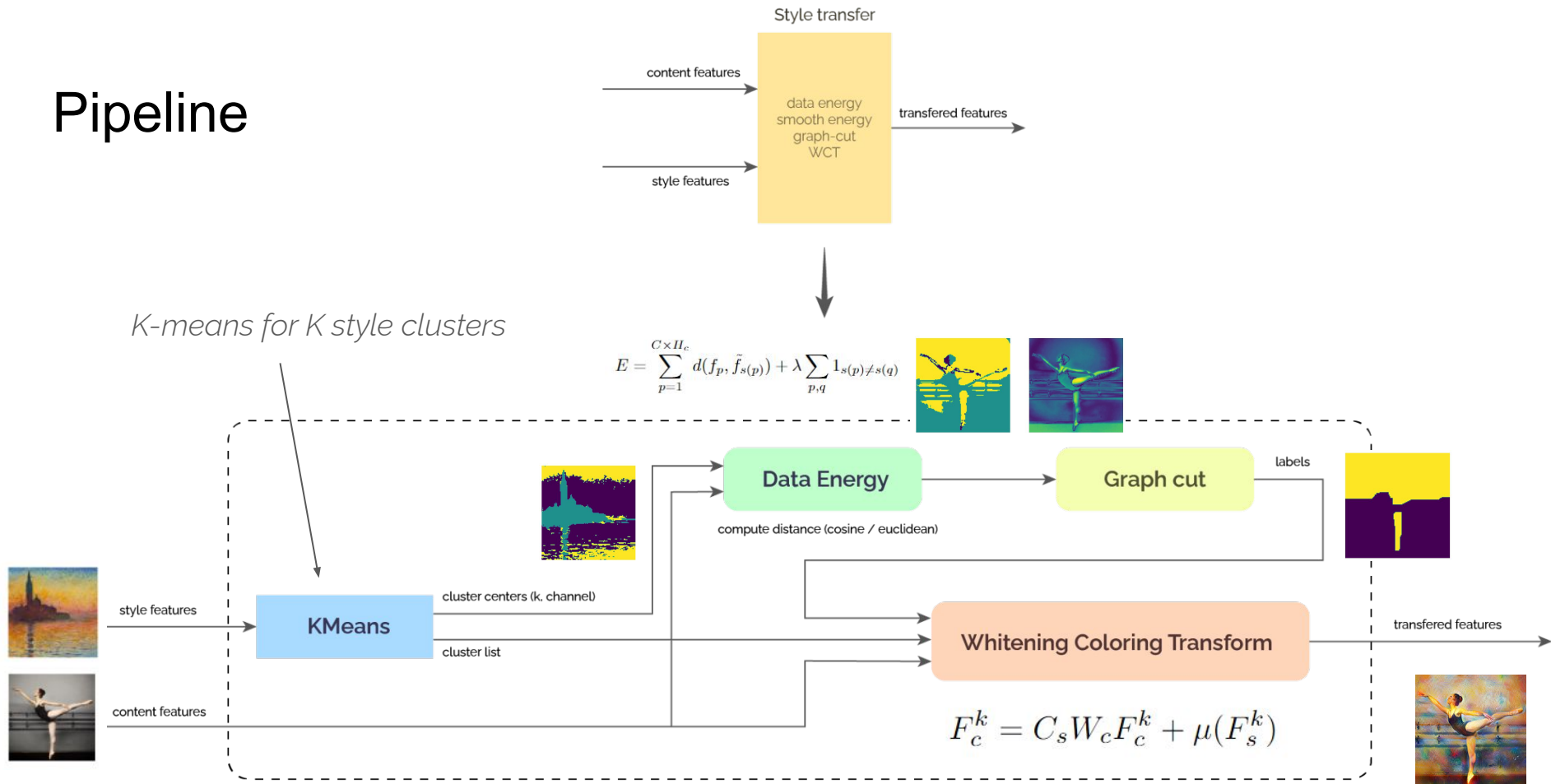
→ MST [1] : solves these issues and allows **multimodality**

multimodality : different styles
applied to different locations in
content image

Pipeline



Pipeline



Alpha expansion

minimize energy function :

$$E = \sum_{p=1}^{C \times H_c} d(f_p, \tilde{f}_{s(p)}) + \lambda \sum_{p,q} 1_{s(p) \neq s(q)}$$

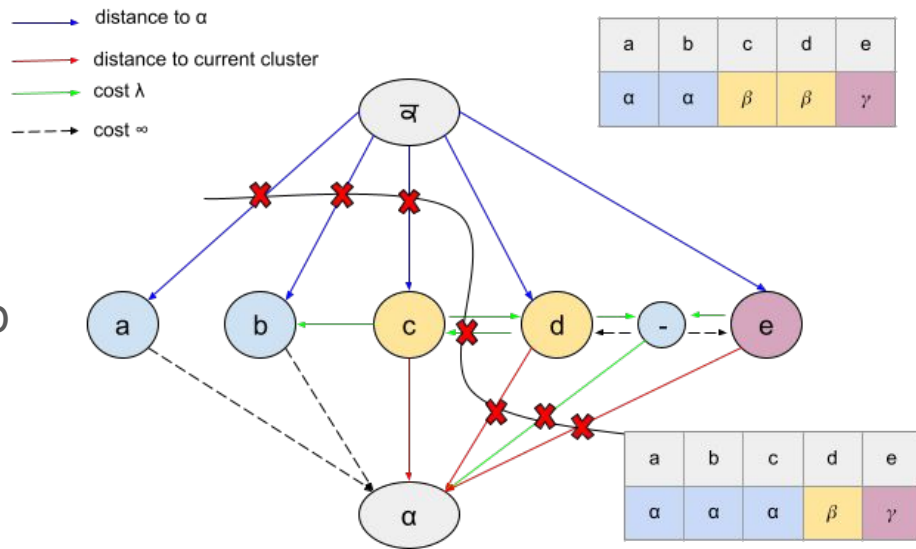
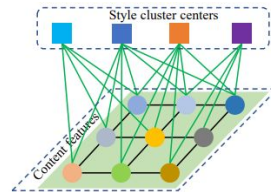
distance to cluster smoothness term

Coding our own algorithm

- ❑ Stochastic : assign random new cluster to random pixel
- ❑ Greedy : assign cluster with smallest total energy to random pixel
- ❑ Alpha expansion :

- choose a label α
- create expansion graph
- graph cut to select nodes to assign α

proven to decrease energy at each step
 → local minima (NP-hard problem) [3]

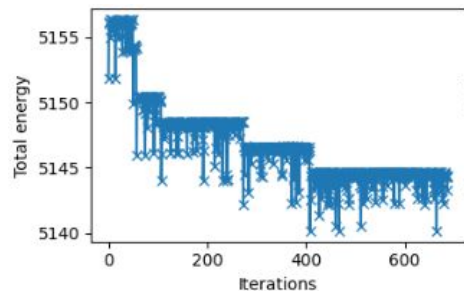


Alpha expansion

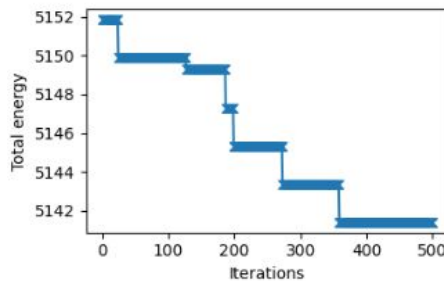
minimize energy function :

$$E = \sum_{p=1}^{C \times H_c} d(f_p, \tilde{f}_{s(p)}) + \lambda \sum_{p,q} 1_{s(p) \neq s(q)}$$

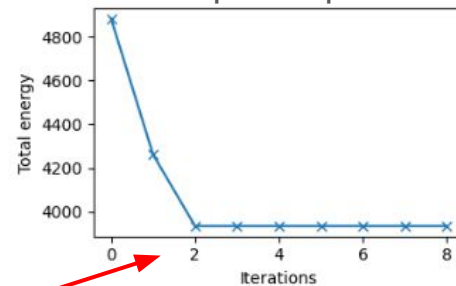
stochastic



greedy



alpha expansion



reached local minima

alpha expansion
choosing λ



$\lambda = 0$



$\lambda = 0.01$



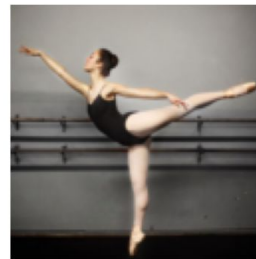
$\lambda = 0.1$



$\lambda = 1$

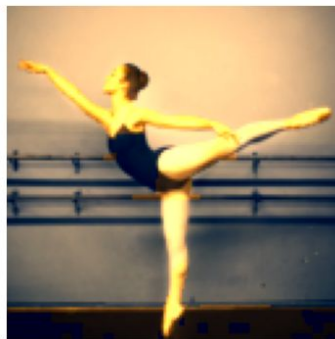
Our experiments

Style transfer of RGB images

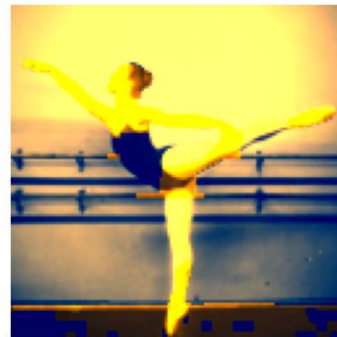


alpha expansion
with $\lambda = 0.01$
choosing α

$$F_{cs}^{l_k} = \alpha_k F_{cs}^{l_k} + (1 - \alpha_k) F_c^{l_k}$$



$\alpha = 0.1$



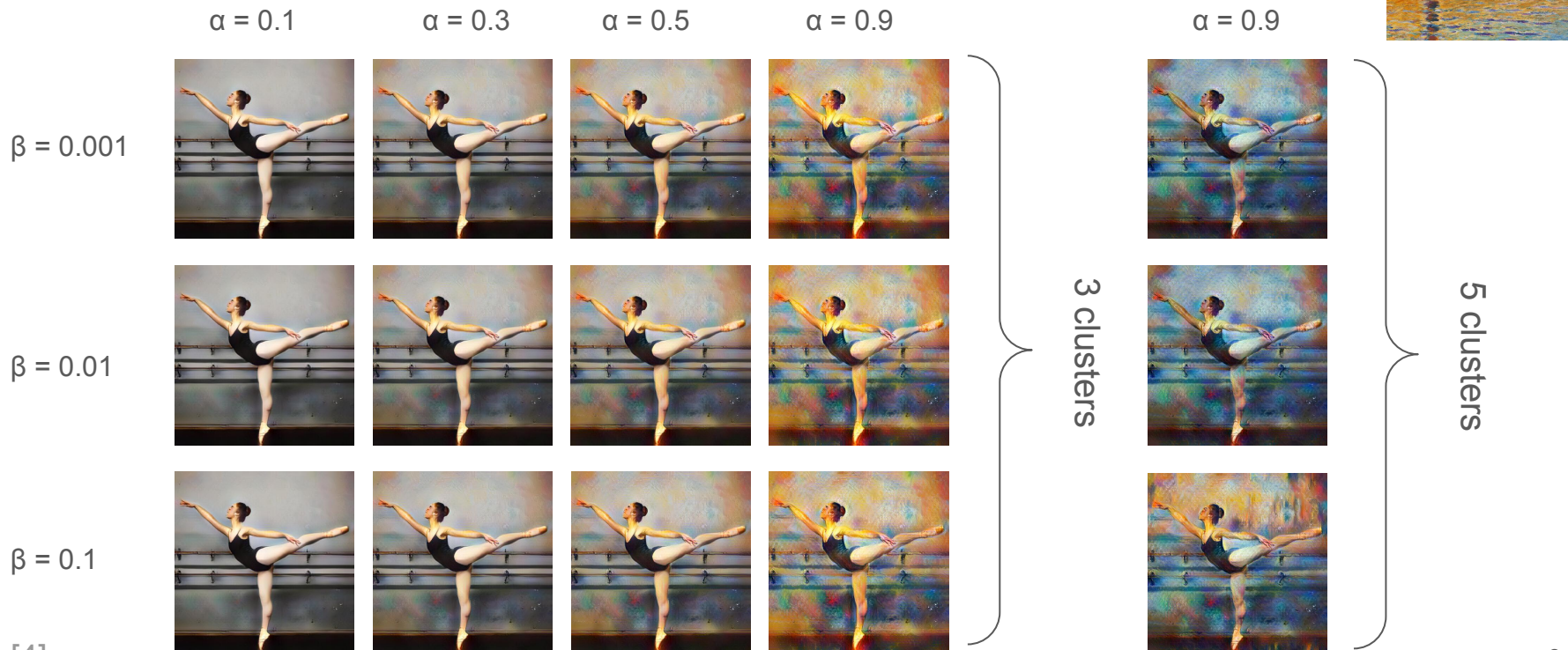
$\alpha = 0.3$



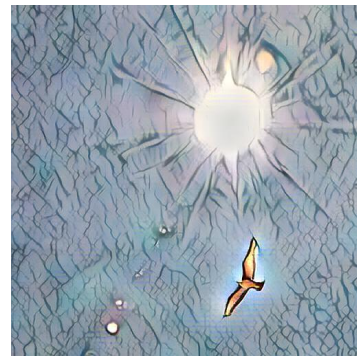
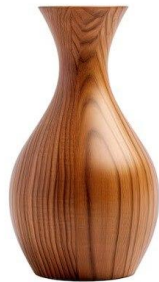
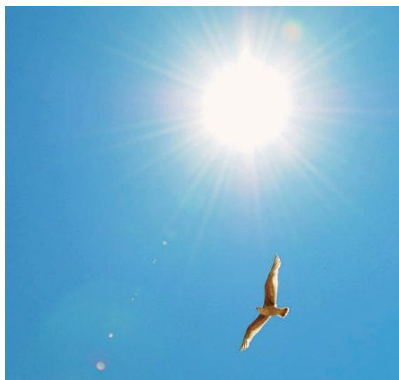
$\alpha = 0.5$

The RGB space is not the best feature space to extract style clusters

Exploring the hyperparameters (full model)



Results



Results



Conclusion

Successively reimplemented the MST algorithm, as well as a study of :

- ❑ alpha expansion algorithm
- ❑ choice of feature space (RGB / VGG)
- ❑ smoothness energy coefficient λ
- ❑ weight of style transfer α

No artefacts and effective multimodality.

Many potential applications : data augmentation, digital art, photo filters (instagram, snapchat, ...), interior design, ...

Drawbacks : must choose α manually depending on context

Thank you for your attention

[1] *Multimodal Style Transfer via Graph Cuts*, Y. Zhang et al. (2020),
<https://arxiv.org/pdf/1904.04443.pdf>

[2] *Understanding Generalized Whitening and Coloring Transform for Universal Style Transfer*, T. Chiu (2019),
https://openaccess.thecvf.com/content_ICCV_2019/papers/Chiu_Understanding_Generalized_Whitening_and_Coloring_Transform_for_Universal_Style_Transfer_ICCV_2019_paper.pdf

[3] *Generalized Fast Approximate Energy Minimization via Graph Cuts: Alpha-Expansion Beta-Shrink Moves*, M. Schmidt et al. (2011), <https://arxiv.org/abs/1108.5710>

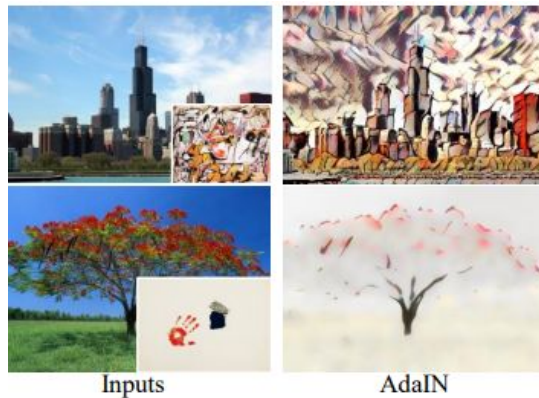
[4] *Very Deep Convolutional Networks for Large-Scale Image Recognition*, K. Simonyan et al. (2014), <https://arxiv.org/abs/1409.1556>

Appendix

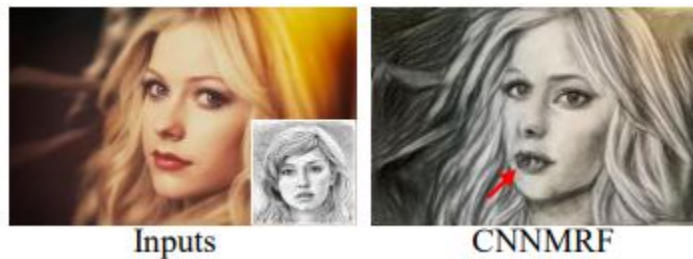
Problems of SOTA

distorsions

wash out



errors



Proposed Method [1]

- ❑ Feature extraction : convolutional network (layer Conv_4_1 of VGG-19) [4]
- ❑ Extract style clusters : K-means for K style clusters
- ❑ (content image pixels) $\leftrightarrow$ (style cluster centers) matching :
using graph cuts, specifically **alpha expansion** [3]
- ❑ Style transfer : feature whitening and coloring (WCT) [2]
- ❑ Decoder : ~mirror of VGG, trained with loss on content and style

Graphcut

Using *networkx* : **Stoer-Wagner algorithm**

recursively merges nodes pair by pair using having only two sets of nodes

we can also use **shortest augmenting path** algorithm to maximize flow (and thus find min-cut via the max-flow min-cut theorem)

Using *igraph* : Stoer-Wagner algorithm

supposed to be faster but we did not observe a difference

WCT

$$F_{cs}^{l_k} = C_s W_c F_c^{l_k} + \mu (F_s^{l_k})$$

Whitening of the content image and coloring using the style clusters

W and C use eigenvalues of the content image and style image respectively

where $W_c = E_c^{l_k} (D_c^{l_k})^{-\frac{1}{2}} (E_c^{l_k})^T$ is a whitening matrix and $C_s = E_s^{l_k} (D_s^{l_k})^{\frac{1}{2}} (E_s^{l_k})^T$ is a coloring matrix. $E_c^{l_k}$ and $D_c^{l_k}$ are diagonal matrix of eigenvalues and the orthogonal matrix of eigenvectors of the covariance matrix $F_c^{l_k} (F_c^{l_k})^T$. For style covariance matrix $F_s^{l_k} (F_s^{l_k})^T$, the corresponding matrices are $E_s^{l_k}$ and $D_s^{l_k}$. The reasons why we choose WCT to transfer features is its robustness and efficiency [24, 23]. More details about whitening and coloring are introduced in [24].

Decoder training

MSE loss on :

- transformed image / content image
- transformed image / style image

(in the feature space)

To train the decoder, we use the pre-trained VGG-19 [37] to compute perceptual loss $l_{total} = l_c + \gamma l_s$, which combines content loss l_c and style loss l_s . We simply set the weighting constant as $\gamma = 10^{-2}$. Inspired by the loss designations in [14, 25, 11], we formulate content loss l_c as

$$l_c = \|\phi_{4,1}(I_c) - \phi_{4,1}(I_{cs})\|_2, \quad (11)$$

where $\phi_{4,1}(\cdot)$ extracts features at layer Conv_4_1 in VGG-19. We then formulate the style loss l_s as

$$l_s = \sum_{i=1}^4 (\|\mu(\phi_{i,1}(I_s)) - \mu(\phi_{i,1}(I_{cs}))\|_2) + \sum_{i=1}^4 (\|\sigma(\phi_{i,1}(I_s)) - \sigma(\phi_{i,1}(I_{cs}))\|_2), \quad (12)$$

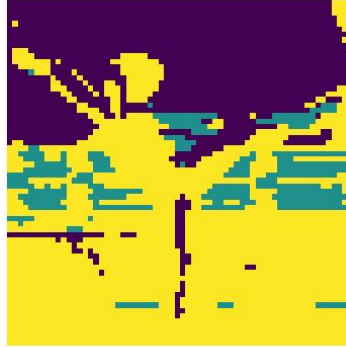
where $\phi_{i,1}(\cdot)$ extracts features at layer Conv_ i _1 in VGG-19. We use $\mu(\cdot)$ and $\sigma(\cdot)$ to compute the mean and standard deviation of the content and style features.

Labeling of Content Features for different λ values

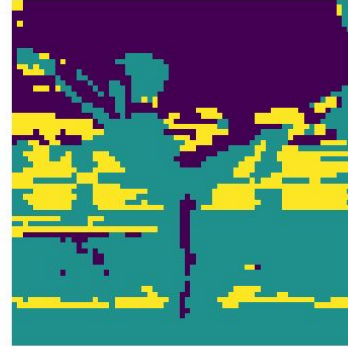
Content Features



Greedy Assignment



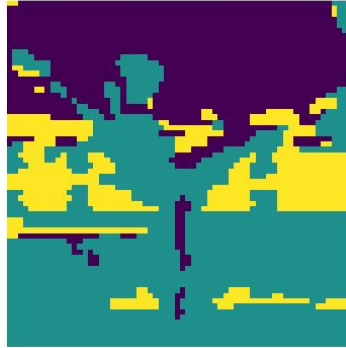
$\lambda=0$



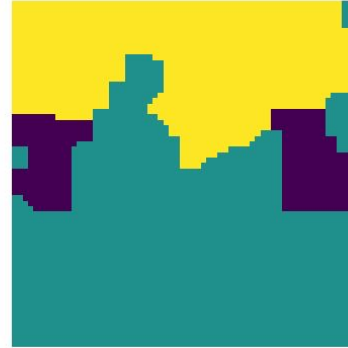
$\lambda=0.001$



$\lambda=0.01$



$\lambda=0.1$



seeing the cluster
match making



distance to chosen
cluster

